

ESOF 322

Software Engineering

LECTURE: 6

J. L. Pach

OUTLINE:

- origin/main
- GitHub Pages
- Markdown

UNIX FILE SYSTEM

UNIX FILE SYSTEM

In Unix/bash, a forward slash `/` is always a directory separator.

```
# this requires directory "left" to already exist
echo "content" > left/right.txt
# create directory first, then file
mkdir -p left
echo "content" > left/right.txt
```

```
"left/right.txt"  →  directory "left" + file "right.txt"
"left/right"      →  directory "left" + file "right" (no extension)
This is a relative path.
The slash ALWAYS means directory separator.
This is the key to understanding Git branches.
```

GIT USES THE FILE SYSTEM AS A DATABASE

Every branch in Git is just a **text file containing one line** – a commit hash.

```
# see what a branch actually is
cat .git/refs/heads/main

# output:
a3f2c11b9e4f2c3d8e7f6a5b4c3d2e1f0a9b8c7d
```

That's it. One hash. That's a branch.

STRUCTURE OF .GIT/REFS

```
tree .git/refs
```

```
.git/refs/
├── heads/      ← ALL local branches
│   ├── main
│   ├── feature/
│   │   ├── tiles
│   │   └── stamina
│   └── fix/
│       └── physics
├── remotes/    ← remote branches
│   └── origin/
│       ├── HEAD
│       └── main
└── tags/       ← tags
    └── v1.0
```

STRUCTURE OF .GIT/REFS

heads/

Contains **all** local branches, not just the default one. Every branch is a file with a commit hash.

remotes/

Contains remote branches after `git fetch`. One subdirectory per remote (`origin`, `upstream`, etc.).

tags/

Contains tags – same mechanism, just a file with a hash.

WHERE IS HEAD?

```
# HEAD is a separate file, not in refs/
cat .git/HEAD

# output when on main:
ref: refs/heads/main

# output in detached HEAD state:
a3f2c11b9e4f2c3d8e7f6a5b4c3d2e1f0a9b8c7d
```

HEAD always tells you **where you are right now**.

Where is `origin`?

```
# origin is NOT in .git/refs
# origin is in .git/config
cat .git/config
```

```
[remote "origin"]
    url = https://github.com/username/repo.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
```

```
origin   =  a name (alias) for a URL
            stored in .git/config
            NOT in .git/refs
You could name it anything:
git remote add superman https://github.com/username/repo.git
git push superman main     ← works perfectly
```

Branch Naming Convention – Why Slashes Work

Because Git uses the file system as a database,
a slash in a branch name creates a **real subdirectory**.

```
git checkout -b feature/tiles # or switch

# Git physically creates:
# .git/refs/heads/feature/tiles
#                └──────────┘ └──┘
#               directory file
```

```
# verify it yourself
tree .git/refs/heads

# output:
heads/
├── main
└── feature/
    └── tiles
```

This is **not just a visual convention**.

It is a real directory structure on disk.

Standard categories used by the industry:

```
feature/    ← new functionality
fix/        ← bug fixes
hotfix/     ← critical production fixes
release/    ← release preparation
chore/      ← maintenance, no code change
```

Git does **not** enforce these categories.

The community invented them because the slash mechanism made it possible.

Full Reference Notation

Every branch has a full address called a **ref**:

```
refs/heads/main                ← local branch main
refs/heads/feature/tiles       ← local branch feature/tiles
refs/remotes/origin/main       ← remote branch main on origin
refs/remotes/origin/feature/tiles  ← remote branch on origin
refs/tags/v1.0                 ← tag
```

Git lets you use shortcuts:

```
# these are identical:
git merge refs/remotes/origin/main
git merge origin/main

# these are identical:
git checkout refs/heads/feature/tiles
git checkout feature/tiles
```

See Everything With One Command

```
# show all refs with their hashes
git show-ref

# output:
a3f2c11 refs/heads/main
b7d4e22 refs/heads/feature/tiles
c9e1f33 refs/remotes/origin/main
d4e5f66 refs/remotes/origin/feature/tiles
e5f7g88 refs/tags/v1.0
```

Summary – The Mental Model

Branch = a file in .git/refs/heads/
containing one commit hash

Remote = a name + URL in .git/config
its branches live in .git/refs/remotes/

HEAD = .git/HEAD
points to your current branch

origin/main = shortcut for refs/remotes/origin/main
two parts separated by slash:
origin = remote name
main = branch name on that remote

feature/tiles = ONE branch name containing a slash
Git creates a real subdirectory
slash here is NOT a remote separator
it is part of the branch name

Commands Cheat Sheet

```
# see all local branches
git branch

# see all remote branches
git branch -r

# see everything
git branch -a

# see all refs with hashes
git show-ref

# see physical structure on disk
tree .git/refs

# see what a branch actually is
cat .git/refs/heads/main

# see where you are
cat .git/HEAD

# see remote URLs and tracking
cat .git/config

# see physical directory structure (bash)
find .git/refs -type f
```

GIT COMMANDS

Understanding the Details

GIT CHECKOUT WAS DOING TOO MUCH

Originally, `git checkout` handled everything:

```
git checkout feature/tiles       # switch branch
git checkout -b feature/tiles    # create and switch branch
git checkout -- main.c           # restore a file
git checkout refs/heads/feature/tiles  # full ref notation
```

One command, too many responsibilities.

This caused confusion, especially for beginners.

Git 2.23 Split checkout Into Two Commands

```
git checkout (old)
    │
    ├── git switch     ← switching and creating branches
    │
    └── git restore    ← restoring files
```

GIT SWITCH – BRANCH OPERATIONS ONLY

```
# switch to existing branch
git switch feature/tiles

# create and switch
git switch -c feature/tiles

# what git switch CANNOT do:
git switch refs/heads/feature/tiles    ✗ does not work
git switch -- main.c                   ✗ does not work
```

GIT RESTORE – FILE OPERATIONS ONLY

```
# discard changes in working directory
git restore main.c

# unstage a file
git restore --staged main.c

# restore file from specific commit
git restore --source HEAD~1 main.c
```

GIT CHECKOUT – STILL WORKS, NOT DEPRECATED

```
# still accepts full ref notation
git checkout refs/heads/feature/tiles    ✓
git checkout feature/tiles               ✓
git checkout -- main.c                   ✓
```

SUMMARY

Operation	Old (still works)	New (recommended)
switch branch	`git checkout branch`	`git switch branch`
create + switch	`git checkout -b branch`	`git switch -c branch`
restore file	`git checkout -- file`	`git restore file`
full ref notation	`git checkout refs/...`	not supported in switch

The Slash – Two Different Meanings

This is one of the most confusing things in Git.

The slash `/` means something **different** depending on context.

CONTEXT 1 – LOCAL REF POINTER (ONE ARGUMENT)

```
git merge origin/main
#         ———————————
#         ONE token
#         local pointer to a remote branch
#         exists as a file at:
#         .git/refs/remotes/origin/main
#
#         origin = remote name  (part of the pointer name)
#         main   = branch name  (part of the pointer name)
#         the slash is PART of the name, not a separator
```

CONTEXT 2 – GIT FETCH ARGUMENTS (TWO ARGUMENTS)

```
git fetch origin main
#         ————————  ————
#         arg1    arg2
#         where   what
#
#         origin = remote name
#         main   = branch name ON that remote
#         there is NO slash here
#         these are two words separated by a space
```

Side by side

```
# ONE argument – local pointer:
git merge origin/main
git log   origin/main
git diff  origin/main

# TWO arguments – remote + branch:
git fetch origin main
git fetch origin feature/tiles
git push  origin feature/tiles
git pull  origin main
```

git fetch – Full Syntax

```
git fetch <remote> <branch>
```

```
# fetch ALL branches from origin
git fetch origin

# fetch only main from origin
git fetch origin main

# fetch only feature/tiles from origin
git fetch origin feature/tiles
#             ——————————————
#             this is the branch name on the remote
#             the branch name contains a slash
#             but it is still ONE argument (arg2)

# fetch from a different remote
git fetch upstream main
git fetch superman  feature/tiles
```

Default Parameters – How Git Fills Them In

When you omit arguments, Git reads `.git/config` to figure out what you meant.

```
# full explicit command:
git fetch origin
git merge origin/main

# shortcut when branch tracking is configured:
git pull            # git knows: fetch origin, merge origin/main
git push             # git knows: push to origin/main

# Git checks .git/config:
cat .git/config
```

```
[remote "origin"]
    url = https://github.com/username/repo.git

[branch "main"]
    remote = origin              ← fetch/push to "origin"
    merge  = refs/heads/main     ← merge with "main"
```

```
# git pull expands to:
git fetch origin
git merge origin/main

# git push expands to:
git push origin main
```

Summary – The Slash Rules

CONTEXT	SLASH MEANING	EXAMPLE
local ref pointer	part of pointer name	origin/main
branch name with category	part of branch name	feature/tiles
git fetch / push / pull	NO slash	git fetch origin main
full ref notation	directory separator	refs/heads/feature/tiles

```
# these mean the same thing:
git merge origin/main
git merge refs/remotes/origin/main

# these are NOT the same syntax:
git merge origin/main      ← ONE argument (local pointer)
git fetch origin main      ← TWO arguments (remote + branch)
```

FETCH

and merge

GIT FETCH AND GIT MERGE – HOW THEY ACTUALLY WORK

The Key Idea

```
fetch   =  update your map of what exists on GitHub
merge   =  apply those changes to your local branch
pull    =  fetch + merge in one command
```

What fetch Actually Does

`git fetch` updates **one specific location** inside `.git/refs/`:

```
git fetch origin

UPDATES:
  .git/refs/remotes/origin/main          ✓ updated to latest hash
  .git/refs/remotes/origin/feature/tiles ✓ updated to latest hash

DOES NOT TOUCH:
  .git/refs/heads/main                   ✗ unchanged
  your working files                     ✗ unchanged
```

Your local branch is untouched.

You have a fresh map of what is on GitHub,

but nothing in your project has changed yet.

FETCH ACCEPTS TWO PARAMETERS

```
# full syntax:
git fetch <remote> <branch>

# fetch ALL branches from origin (one argument):
git fetch origin

# fetch only one specific branch (two arguments):
git fetch origin main
git fetch origin feature/tiles

# if you provide only the remote name
# Git fetches ALL branches from that remote
# if you provide remote + branch
# Git fetches only that one branch
```

What merge Does

`git merge origin/main` takes the hash stored in `.git/refs/remotes/origin/main` and applies it to your local branch.

```
BEFORE merge:

.git/refs/
├── heads/
│   └── main        ← old hash (your local state)
└── remotes/
    └── origin/
        └── main    ← new hash (what is on GitHub)
AFTER git merge origin/main:
.git/refs/
├── heads/
│   └── main        ← updated to new hash ✓
└── remotes/
    └── origin/
        └── main    ← unchanged
```

Three Possible Outcomes of merge

OUTCOME 1 – FAST-FORWARD (NOTHING TO MERGE)

This happens when you have not added any new commits locally since the last time you were in sync with GitHub.
GitHub has moved forward, you have not.

```
BEFORE:
local main:   A → B → C
                    ↑
                  HEAD

origin/main:  A → B → C → D → E
                        ↑
                    origin/main
```

```
git fetch origin
git merge origin/main

# Git says: "Fast-forward"
```

```
AFTER:
local main:   A → B → C → D → E
                        ↑
                   HEAD/main
                   origin/main

No new commit created.
Git simply moves your HEAD pointer forward to E.
D and E now exist in your local history.
```

OUTCOME 2 – MERGE COMMIT (BOTH SIDES HAVE NEW COMMITS)

This happens when you and someone else both added commits
since the last common point.
Different files were edited, so Git can merge automatically.

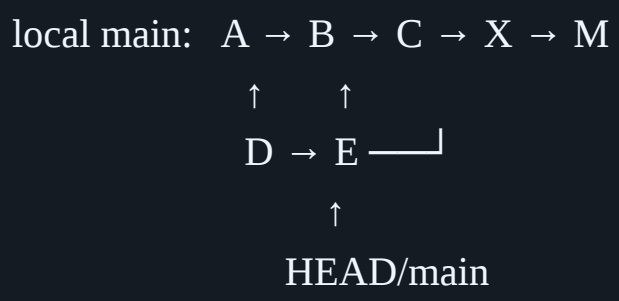
```
BEFORE:
local main:   A → B → C → X
                    ↑   ↑
                    |  HEAD/main
                    |
origin/main:  A → B → C → D → E
                        ↑
                     origin/main

C is the last common commit (common ancestor)
X is your commit
D and E are commits from GitHub
```

```
git fetch origin
git merge origin/main

# Git says: "Merge made by the 'recursive' strategy"
```

AFTER:



M = merge commit

M has two parents: X and E

M now exists only locally – not on GitHub yet

git push will send M to GitHub

no --force needed because M contains E as a parent

OUTCOME 3 – CONFLICT (SAME FILE EDITED IN BOTH PLACES)

This happens when you and someone else edited
the **same lines in the same file**.
Git cannot decide which version to keep.

```
BEFORE:
local main:   A → B → C → X    ← X edits line 42 of main.c
                    ↑   ↑
                    |  HEAD/main
                    |
origin/main:  A → B → C → D → E ← E edits line 42 of main.c
                        ↑
                      origin/main

C is the last common commit
both X and E touch the same line
```

```
git fetch origin
git merge origin/main

# Git stops:
# CONFLICT (content): Merge conflict in main.c
# Automatic merge failed; fix conflicts and then commit the result.
```

```
AFTER fetch + failed merge attempt:

local main:   A → B → C → X
                    ↑
                  HEAD/main  ← DID NOT MOVE

origin/main has D and E locally but they are not merged yet
Git is waiting for you to resolve the conflict
```

Git marks the conflict inside the file:

```
<<<<<< HEAD
    stamina -= 15;        ← your version (commit X)
=======
    stamina -= 20;        ← version from GitHub (commit E)
>>>>>> origin/main
```

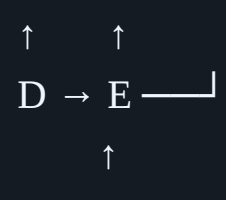


You must:

1. open the file
2. remove the conflict markers
3. keep the correct code
4. git add main.c
5. git commit

AFTER you resolve and commit:

local main: A → B → C → X → M



Same result as Outcome 2
but M was created manually by you
not automatically by Git

GIT PULL = GIT FETCH + GIT MERGE

```
# these two blocks are identical:

# block 1 – explicit:
git fetch origin
git merge origin/main

# block 2 – shortcut:
git pull origin main
```

```
# when tracking is configured (after git push -u origin main):
git pull
# Git reads .git/config and expands to:
# git fetch origin
# git merge origin/main
```

SUMMARY

COMMAND	UPDATES	TOUCHES YOUR FILES?
git fetch origin	refs/remotes/origin/*	NO
git fetch origin main	refs/remotes/origin/main	NO
git merge origin/main	refs/heads/main	YES
git pull origin main	both of the above	YES

fetch → updates .git/refs/remotes/ (your map of GitHub)
merge → updates .git/refs/heads/ (your local branch)
pull → does both in one step

OUTCOME COMPARISON

OUTCOME	TRIGGER	RESULT
Fast-forward	only GitHub has new commits	HEAD moves forward
		no new commit created
Merge commit	both sides have new commits	new commit M created
	different files edited	automatically by Git
		two parents: your commit + E
Conflict	both sides edited	Git stops
	same lines same file	HEAD does not move
		you resolve manually
		then commit M yourself

GITHUB

Pages

SETTING UP GITHUB PAGES WITH MARKDOWN

Create a Repository

- Go to **GitHub** and click **New Repository**.
- Give it a name (e.g., assignment04).
- Initialize with a README.md.

SETTING UP GITHUB PAGES WITH MARKDOWN

Add a Static Markdown Page

- Write something simple in README.md :
- or index.md

```
# Welcome to Assignment 04
This is my first GitHub Pages project.
```

SETTING UP GITHUB PAGES WITH MARKDOWN

Add Jekyll Configuration

- Create a file called _config.yml in the repo root.
- Example content:
- theme gallery: GitHub Pages Themes

https://docs.github.com/en/pages/setting-up-a-github-pages-site-with-jekyll/adding-a-theme-to-your-github-pages-site-using-jekyll

```
title: Assignment 04
show_downloads: false
theme: jekyll-theme-minimal
description: MarkDown
```

SETTING UP GITHUB PAGES WITH MARKDOWN

Configure GitHub Pages

- ⚒ Go to Settings → Pages.
- ⚒ Under Build and Deployment, select Deploy from branch.
- ⚒ Choose main and / (root).
- ⚒ Save — your site will be live at:

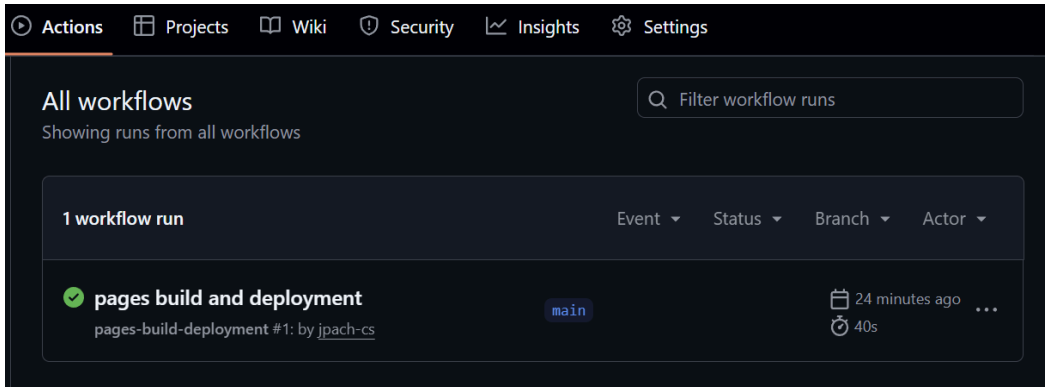
https://username.github.io/assignment02/

SETTING UP GITHUB PAGES WITH MARKDOWN

Check Your Page

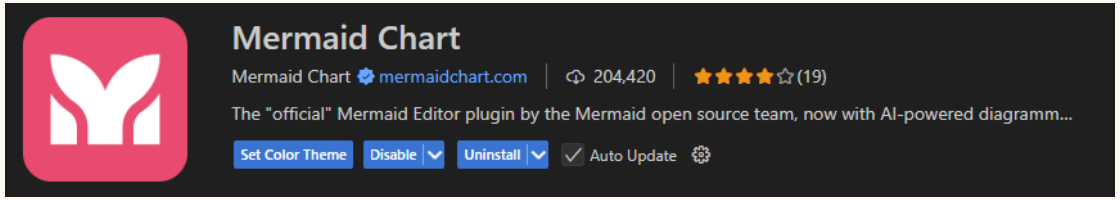
After updating the changes in the `_config.yml` and `README.md` files, GitHub's server needs to generate the HTML, CSS, and other files based on them. If the page does not appear after refreshing the link, there are two possibilities: either the build is still in progress (marked in yellow, so you need to wait a moment), or there is a syntax error, in which case the last action will fail.

- Go to Actions
- Visit your site link again and refresh.
- You should now see the applied theme.

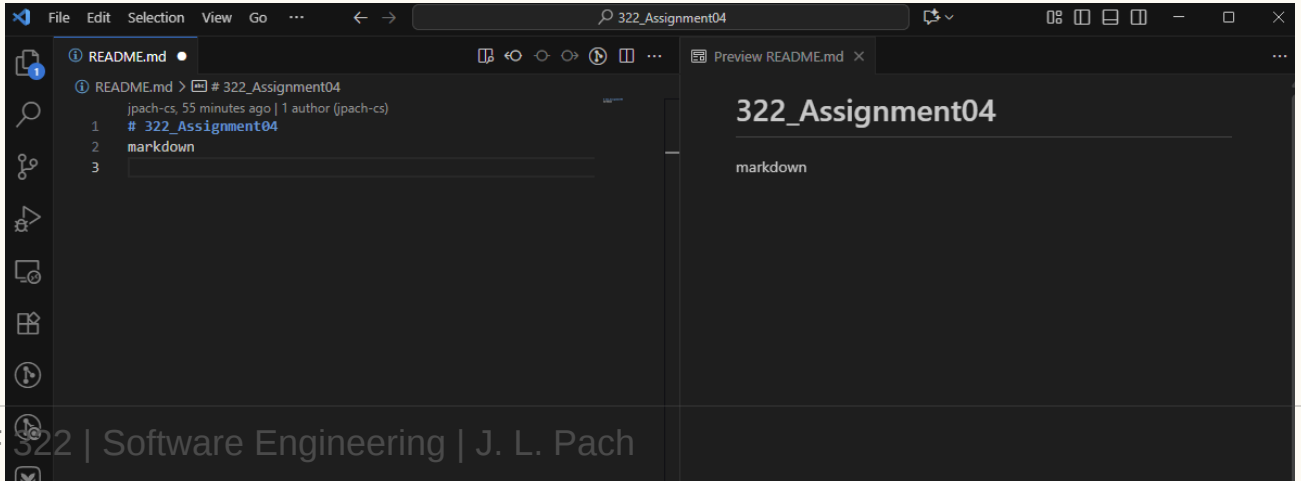


CLONE YOUR REPO

- After creating the repository, clone it locally, open the folder in Visual Studio Code,
- install the Mermaid Chart extension,



- right-click on README.md and select *Preview*. You can now split the window to edit and preview at the same time.



MARKDOWN

INTRODUCTION TO MARKDOWN

What is Markdown?

- A **lightweight markup language** for formatting text.
- Created by **John Gruber in 2004** with help from Aaron Swartz.
- Goal: make documents easy to read in plain text, but convertible to HTML.

INTRODUCTION TO MARKDOWN

Markdown and CommonMark

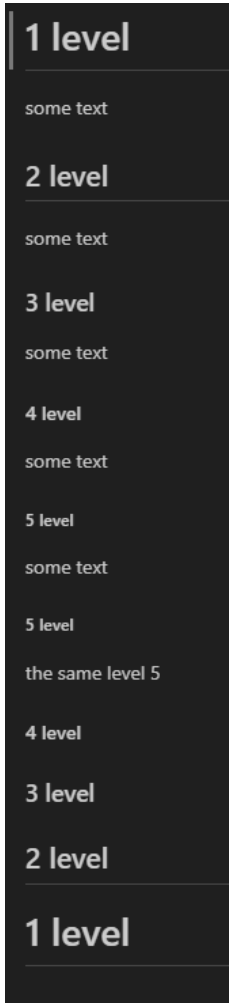
- CommonMark = a formal specification of Markdown (2014).
- Ensures consistency across tools.
- GitHub uses GitHub Flavored Markdown (GFM) → extends CommonMark with:
  - Task lists (- [ ], - [x])
  - Tables
  - Code fencing with language highlighting
  - Mentions (@username) and issue links

CORE MARKDOWN FEATURES

- Headings: #, ##, ###
- Lists:
  - Unordered: - item
  - Ordered: 1. item
- Links: [text](url)
- Images: ![alt](url)
- Tables ||
- Code:
  - Inline: `code`
  - Block: triple backticks

CORE MARKDOWN FEATURES - HEADINGS 1/2

```
# 1 level
some text
## 2 level
some text
### 3 level
some text
#### 4 level
some text
##### 5 level
some text
##### 5 level
the same level 5
#### 4 level
### 3 level
## 2 level
# 1 level
```



CORE MARKDOWN FEATURES - HEADINGS 2/2

- Headings: `#`, `##`, `###`
- Headings in Markdown work very similarly to headings in MS Word.
- By default, Markdown supports 5 levels of headings.

However, there is a general rule of text organization that applies independently of Word or Markdown:

If you divide text into chapters and subchapters, a subchapter cannot consist of only one item, because such a division makes no sense. On the other hand, if subchapters differ drastically in length or content, it usually means that the structure was poorly designed. There is also an unwritten rule that you should avoid using more than three levels of subdivision.

CORE MARKDOWN FEATURES - LISTS 1/2

- **Lists in Markdown** work similarly to those in MS Word. There are two main types: **ordered** and **unordered**.
- For unordered lists, Markdown expects a **dash (-)** before each item.
- For ordered lists, you use **numbers followed by a dot**.
- **Keep in mind**: in Markdown it doesn't matter what number you type—Markdown will **automatically renumber** the items correctly in sequence.

CORE MARKDOWN FEATURES - LISTS 1/2

```
my unordered list:
- 1 item
- 2 item

my unordered list:
1. 1 item
1. 2 item

but:

my unordered list:
1. 1 item
3. 2 item
```

my unordered list:

- 1 item
- 2 item

my unordered list:

1. 1 item
2. 2 item

but:

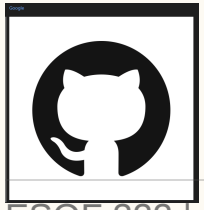
my unordered list:

1. 1 item
2. 2 item

CORE MARKDOWN FEATURES - LINKS AND IMAGES

- `[text](url)`:
  - text → the word that will be clickable
  - url → the website address
- `![alt](url)`:
  - alt → alternative text (displayed if the image cannot load)
  - url → the path or link to the image

```
[Google](https://www.google.com)
![Logo](https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png)
```



CORE MARKDOWN FEATURES - TABLES 1/2

- `|` separates columns
- `---` defines a header row
- `:` can align text (left, center, right)

```
| Column 1 | Column 2 | Column 3 |
|----------|----------|----------|
| Value 1  | Value 2  | Value 3  |
| A        | B        | C        |
```

Column 1	Column 2	Column 3
Value 1	Value 2	Value 3
A	B	C

CORE MARKDOWN FEATURES - TABLES 2/2

```
| Column 1 | Column 2 | Column 3 |
|:---------|:--------:|---------:|
| Value 1  | Value 2  | Value 3  |
| A        | B        | C        |
```

Left	Center	Right
a	b	c

> Note: **Markdown itself does not support cell merging**; for that, you need to embed `HTML` within the .md file.

CORE MARKDOWN FEATURES - CODE

- Inline code: `` `code` ``
- Code block (multi-line) -
  ```language_label
  language your code here ```

````
```c
def hello():
    print("Hello Markdown!")
```
````

```c
def hello():
    print("Hello Markdown!")
```

Note: After three tildes, you can inform Markdown what language the code is in for syntax highlighting, but this only works for code blocks.
```

# THANK YOU!